CARESYNC MEDCARE — TECHNICAL REPORT

Complete Engineering Blueprint: Prototype through Production Architecture

LEAD DEVELOPER
Bafokeng Khoali

TARGET PLATFORM
Multi-Tenant SaaS

COMPLIANCE
HIPAA 2026 / HITECH

DATE
October 2026

1. EXECUTIVE SUMMARY & HARDWARE CONSTRAINTS

CareSync MedCare is a cloud-native, multi-tenant integrated healthcare platform supporting six primary clinical domains: **Mental Health, Dental, Optometry, Nursing, Wellness, and Community Outreach**. The software engineering model is structured to seamlessly transition from a zero-cost prototype phase to an enterprise-grade Azure/Google Cloud hybrid production environment while strictly enforcing HIPAA/HITECH 2026 data privacy standards.

CRITICAL WORKSTATION CONSTRAINT (HP INSPIRON 13 — i7-7500U / 8GB RAM / Intel HD 620): Your development laptop is configured purely as a *lightweight IDE workstation and orchestration terminal*. **Do NOT run local Docker containers, local PostgreSQL instances, Redis servers, or RabbitMQ brokers.** Local resource consumption must not exceed ~4GB RAM (VS Code ~1GB, Chrome DevTools ~2GB, Java Language Server ~500MB). All execution, builds, databases, and message queuing must be offloaded directly to cloud runtimes (Render, Supabase, Firebase, CloudAMQP, Vercel).

2. PROTOTYPE ARCHITECTURE (FREE-TIER STACK)

The prototype stack leverages cloud free tiers to achieve zero operational expenditure while validating end-to-end multi-tenant flows, asynchronous event distribution, and fine-grained authorization.

Layer	Technology Stack	Hosting Provider	Free Tier Capacity & Operational Limits
Frontend	React 18 + Vite + TypeScript	Vercel	100 GB Bandwidth/mo, Automated CI/CD CD Pipeline
API Gateway / Microservices	Spring Boot 3.2 (Java 21 OpenJDK)	Render	750 execution hrs/mo, auto-sleeps after 15m inactivity
Relational DB (PHI)	PostgreSQL 15 (UUID, Row Level Security)	Supabase	500 MB DB storage, 2 GB egress, direct SSL access
Document DB (Clinical Notes)	Google Cloud Firestore	Firebase	1 GB storage, 50K reads/day, real-time SDK integration
Identity & Auth	Firebase Auth + Custom Claims JWT	Firebase	50,000 Monthly Active Users (MAU), OIDC Support
Cache Layer	Redis 7 (In-Memory Data Structure)	Render	25 MB RAM capacity, key eviction configured
Message Queue	RabbitMQ 3.12 (AMQP Protocol)	CloudAMQP	1,000,000 messages/mo (Little Lemur Shared Shared Cluster)
Object Storage	Supabase Storage (S3 API Compatible)	Supabase	1 GB bucket capacity with AES-256 server-side encryption
AI/ML Workspace	Python 3.11 + scikit-learn + Jupyter	Google Colab	Ephemeral GPU (Tesla T4) runtime for training execution

2.2 MICROSERVICE DECOMPOSITION & PORT ALLOCATION

Service 1: `caresync-api-gateway` (Port 8080)
Spring Cloud Gateway instance. Handles JWT validation with Firebase Admin SDK, rate-limiting using Bucket4j (100 req/min per IP), CORS configuration, and SSL termination.

Service 2: `caresync-patient-service` (Port 8081)
Manages tenant-isolated patient demographics, insurance profiling, emergency contacts, and search indexes via Supabase PostgreSQL.

Service 3: `caresync-clinical-service` (Port 8082)
Handles appointment scheduling, provider availability matrices, and metadata linkage between PostgreSQL and Firestore Clinical Notes.

Service 4: `caresync-billing-service` (Port 8083)
Generates invoices, tracks payment statuses, caches billing codes in Redis, and publishes asynchronous audit/invoice events to RabbitMQ.

2.3 POSTGRESQL SCHEMA DEFINITION (SUPABASE SQL DDL)

```
-- ================================================================================
-- CARESYNC MEDCARE - PROTOTYPE RELATIONAL DATABASE SCHEMA (POSTGRESQL 15)
-- ================================================================================
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- MULTI-TENANCY CORE TABLE
CREATE TABLE tenants (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(255) NOT NULL,
    subdomain VARCHAR(100) UNIQUE NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    is_active BOOLEAN DEFAULT TRUE
);

-- PATIENT DEMOGRAPHICS (PHI ENCRYPTED AT REST)
CREATE TABLE patients (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    date_of_birth DATE NOT NULL,
    gender VARCHAR(20),
    email VARCHAR(255),
    phone VARCHAR(20),
    address TEXT,
    emergency_contact_name VARCHAR(200),
    emergency_contact_phone VARCHAR(20),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    UNIQUE(tenant_id, email)
);
CREATE INDEX idx_patients_tenant ON patients(tenant_id);
CREATE INDEX idx_patients_name ON patients(tenant_id, last_name, first_name);

-- HEALTHCARE PROVIDERS / CLINICAL STAFF
CREATE TABLE providers (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    user_id VARCHAR(128) NOT NULL, -- Direct Reference to Firebase Auth UID
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    specialty VARCHAR(100) NOT NULL CHECK (specialty IN ('DENTAL', 'MENTAL_HEALTH', 'OPTOMETRY', 'NURSIN
G', 'WELLNESS', 'COMMUNITY')),
    license_number VARCHAR(100),
    is_active BOOLEAN DEFAULT TRUE,
```

2.4 FIRESTORE SCHEMA & SECURITY RULES (CLINICAL NOTES)

Unstructured clinical data (progress notes, psychiatric assessments, optOMETRY imaging metadata) is stored in Firebase Firestore to separate heavy document payloads from relational transactional databases.

```
// Firestore Collection: /clinical_notes/{note_id}
{
  "tenant_id": "8f3b2a1c-90d1-4e22-a8b3-1a2b3c4d5e6f",
  "patient_id": "a1b2c3d4-e5f6-7a8b-9c0d-1e2f3a4b5c6d",
  "provider_id": "f9e8d7c6-b5a4-3f2e-1d0c-9b8a7f6e5d4c",
  "appointment_id": "c1d2e3f4-5a6b-7c8d-9e0f-1a2b3c4d5e6f",
  "specialty": "MENTAL_HEALTH",
  "note_type": "SOAP_NOTE",
  "soap_data": {
    "subjective": "Patient reports persistent anxiety and insomnia for 2 weeks.",
    "objective": "Hamilton Anxiety Rating Scale score: 22 (Moderate).",
    "assessment": "Generalized Anxiety Disorder (ICD-10 F41.1).",
    "plan": "Initiate CBT therapy twice weekly. Follow up in 14 days."
  },
  "attachments": [{"name": "scan.pdf", "url": "https://supabase.co/storage/v1/..."}],
  "created_at": "2026-10-25T10:30:00Z"
}
```

```
// FIRESTORE SECURITY RULES (ENFORCING TENANT & SPECIALTY ISOLATION)
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /clinical_notes/{noteId} {
      allow read, write: if request.auth != null
        && request.auth.token.tenant_id == resource.data.tenant_id
        && (request.auth.token.role == 'ADMIN'
            || (request.auth.token.role == 'PROVIDER'
                && request.auth.token.specialty == resource.data.specialty));
    }
  }
}
```

3. PRODUCTION ARCHITECTURE (AZURE ENTERPRISE + GOOGLE CLOUD AI/ML)

When transitioning from prototype to full commercial scale, CareSync migrates core transactional runtimes to **Microsoft Azure** while deploying specialized AI/ML workflows onto **Google Cloud Platform (GCP)**.

Why Azure for Core Infrastructure?
Compliance & Interoperability: Includes HIPAA BAA out-of-the-box. Offers managed *Azure Health Data Services* providing turn-key HL7 FHIR r4 servers and DICOM medical imaging services seamlessly integrated with Microsoft Entra ID.

Why Google Cloud for Healthcare AI?
Vertex AI & BigQuery: GCP Vertex AI provides HIPAA-compliant access to Gemini enterprise foundation models for automated clinical note summarization, coding assistance, and population health analytics.

HIPAA 2026 Mandate — Data Isolation Model: Production deployments migrate from shared-database single-tenant schemas to a **Database-per-Tenant model using Azure SQL Elastic Pools**. This guarantees zero risk of cross-tenant data leakage at the storage tier while maintaining cost efficiency.

3.3 PRODUCTION SECURITY ARCHITECTURE & HIPAA COMPLIANCE MATRIX

As the Lead Developer and Security Specialist, security is built directly into the container configurations, network routing rules, and CI/CD automated gates rather than bolted on post-development.

Security Control Domain	Prototype Implementation (Free Tier)	Production Enterprise Standard (Azure + GCP)
Identity & Access Control	Firebase Auth + JWT Custom Claims (Tenant ID, Role, Specialty)	Microsoft Entra ID (Azure AD) + MFA + Conditional Access Policies
Secrets Management	Render Environment Variables / Github Secrets	Azure Key Vault with Hardware Security Module (HSM) dedicated backing
Data-at-Rest Encryption	Supabase default AES-256 (Cloud Provider Keys)	Transparent Data Encryption (TDE) with Customer-Managed Keys (BYOK)
Data-in-Transit Encryption	TLS 1.2+ forced on Vercel, Render, and Supabase endpoints	TLS 1.3 enforced globally via Azure Front Door & Private Endpoints
Network Security	Public Cloud API endpoints protected via API Keys & Bearer Tokens	Azure Virtual Network (VNet) isolation, Private Link, Zero Public IPs
Audit Logging	PostgreSQL `audit_logs` table with JSONB delta snapshots	Immutable Azure Log Analytics Workspace + Azure Sentinel SIEM integration
DDoS & WAF Protection	Cloudflare DNS Basic Protections	Azure Front Door WAF with OWASP Top 10 automated block rulesets
Interoperability Standards	Custom JSON REST APIs	HL7 FHIR v4 REST API + DICOMweb API via Azure Health Data Services

3.4 THREAT MODELING & PENETRATION TESTING STRATEGY

To maintain continuous compliance, the development pipeline incorporates automated static and dynamic security checks, ensuring no vulnerability reaches staging or production environments.

SAST & Dependency Analysis

GitHub Actions execute *Snyk* and *Trivy* scans on every pull request to detect vulnerable Java dependencies, NPM packages, and Docker file misconfigurations.

DAST / Penetration Scans

Automated *OWASP ZAP* scans target Render staging endpoints nightly to detect SQL Injection, Broken Object Level Authorization (BOLA), and CORS misconfigurations.

BOLA Defense Validation

Every controller endpoint enforces double-check tenant validation: validating `JWT.tenant_id` matches `{tenant_id}` route params before executing database queries.

4. COMPLETE DRAW.IO XML ARCHITECTURE SPECIFICATIONS

Below are the raw XML file definitions for the system architecture diagrams. These can be saved as `.drawio` files or pasted directly into draw.io for editing and export.

4.1 Diagram Definition: Prototype Architecture (Free Tier)

```
<mxfile host="Electron" version="21.1.2" type="device">
  <diagram id="proto-arch" name="Prototype Architecture">
    <mxGraphModel dx="1422" dy="794" grid="1" gridSize="10" guides="1" tooltips="1" connect="1" arrows
="1" page="1" pageWidth="1400" pageHeight="900">
      <root>
        <mxCell id="0" />
        <mxCell id="1" parent="0" />
        <mxCell id="title" value="CareSync Prototype Architecture (Free Tier)" style="text;html=1;strokeC
olor=none;fillColor=none;align=center;fontSize=20;fontStyle=1;" vertex="1" parent="1">
          <mxGeometry x="400" y="20" width="600" height="40" as="geometry" />
        </mxCell>
        <mxCell id="vercel" value="Frontend (React + Vite + TS)
Vercel Free Tier" style="rounded=1;whiteSpace=wrap;html=1;fillColor=#dae8fc;strokeColor=#6c8ebf;fontStyle
=1;" vertex="1" parent="1">
          <mxGeometry x="550" y="100" width="300" height="70" as="geometry" />
        </mxCell>
        <mxCell id="render-gw" value="API Gateway (Spring Boot)
Render Free Tier" style="rounded=1;whiteSpace=wrap;html=1;fillColor=#fff2cc;strokeColor=#d6b656;fontStyle
=1;" vertex="1" parent="1">
          <mxGeometry x="550" y="220" width="300" height="70" as="geometry" />
        </mxCell>
        <mxCell id="svc-patient" value="Patient Service
(Render)" style="rounded=1;whiteSpace=wrap;html=1;fillColor=#d5e8d4;strokeColor=#82b366;" vertex="1" pare
nt="1">
          <mxGeometry x="150" y="370" width="180" height="60" as="geometry" />
        </mxCell>
        <mxCell id="svc-clinical" value="Clinical Service
(Render)" style="rounded=1;whiteSpace=wrap;html=1;fillColor=#d5e8d4;strokeColor=#82b366;" vertex="1" pare
nt="1">
          <mxGeometry x="400" y="370" width="180" height="60" as="geometry" />
        </mxCell>
        <mxCell id="svc-billing" value="Billing Service
(Render)" style="rounded=1;whiteSpace=wrap;html=1;fillColor=#d5e8d4;strokeColor=#82b366;" vertex="1" pare
nt="1">
          <mxGeometry x="650" y="370" width="180" height="60" as="geometry" />
        </mxCell>
        <mxCell id="supabase" value="Supabase PostgreSQL
(Relational PHI Data)" style="shape=cylinder3;whiteSpace=wrap;html=1;fillColor=#f8cecc;strokeColor=#b8545
0;" vertex="1" parent="1">
          <mxGeometry x="150" y="500" width="180" height="80" as="geometry" />
        </mxCell>
        <mxCell id="firestore" value="Firebase Firestore
(Clinical Notes)" style="shape=cylinder3;whiteSpace=wrap;html=1;fillColor=#f8cecc;strokeColor=#b85450;" v
ertex="1" parent="1">
          <mxGeometry x="400" y="500" width="180" height="80" as="geometry" />
        </mxCell>
        <mxCell id="rabbit" value="RabbitMQ (CloudAMQP)
Async Queue" style="rounded=1;whiteSpace=wrap;html=1;fillColor=#e1d5e7;strokeColor=#9673a6;" vertex="1" p
arent="1">
          <mxGeometry x="650" y="510" width="180" height="60" as="geometry" />
        </mxCell>
      </root>
    </mxGraphModel>
  </diagram>
</mxfile>
```

4.2 Diagram Definition: Production Azure + GCP Architecture

```
<mxfile host="Electron" version="21.1.2" type="device">
```

5. DEVELOPMENT WORKFLOW & SOLO-DEVELOPER STRATEGY

Operating as a solo developer requires strict adherence to automated cloud pipelines and git-driven deployment workflows to avoid exceeding local hardware thresholds while maintaining code quality.

5.1 Workstation Workspace Structure

```
caresync_medcare/
├── .github/workflows/          # CI/CD GitHub Action pipelines (Snyk, Build, Deploy)
├── backend/
│   ├── api-gateway/            # Spring Cloud Gateway (Port 8080)
│   ├── patient-service/        # Demographics & Tenant isolation (Port 8081)
│   ├── clinical-service/       # Appointments & Firestore Metadata (Port 8082)
│   └── billing-service/        # Invoices, Redis Cache & RabbitMQ (Port 8083)
├── frontend/                   # React 18 + Vite + TypeScript (Hosted on Vercel)
├── ai-ml/                      # Python notebooks (Executed in Google Colab)
└── docs/                       # Draw.io XML files & OpenAPI/Swagger specifications
```

5.2 Solo Development Execution Loop

1. Cloud Execution Model
Do NOT compile microservice Docker images locally. Push commits to GitHub features branches; Render and Vercel pick up changes and execute remote container builds automatically.

2. Remote Debugging Strategy
Connect VS Code directly to remote Supabase and Firestore instances using environment variable profiles (`.env.local`). Stream logs directly via Render CLI tools.

6. DEVELOPER HANDOFF & MAINTENANCE GUIDELINES

When onboarding additional engineering staff or transitioning this codebase to future development teams, follow these mandatory conventions:

- **Repository Source of Truth:** Never apply inline schema changes in Supabase dashboard. All database modifications MUST be authored as Flyway or Liquibase SQL migration scripts stored in `/backend/{service}/src/main/resources/db/migration`.
- **Secrets Isolation:** Never commit API keys, service account JSON files, or database passwords to Git. Use environment secrets injected via Vercel, Render, or Azure Key Vault.
- **Testing Zero-Trust:** Every new REST API endpoint must contain an accompanying JUnit integration test verifying that a user from `Tenant A` receives an HTTP 403 Forbidden when requesting resources belonging to `Tenant B`.